Lab Overview

- **Platform:** HackerDNA
- **Difficulty:** Very Easy
- **Category:** Web & API Security
- **Objective:** Exploit a cookie poisoning vulnerability in a corporate portal to elevate privileges from `guest` to `admin` and capture the flag.

Reconnaissance & Initial Access

1. **Accessing the Portal:** Navigating to the lab URL opens a corporate login portal with the credentials pre-filled.
2. **Authentication:** Logging in with the default credentials (`guest` / `guest`) directs us to the user dashboard containing a few non-functional quick links.

Vulnerability Analysis

Inspecting the browser's Developer Tools (`F12` or `Ctrl+Shift+I`), we navigate to **Storage > Cookies** and locate the active session cookie.
The cookie value appears to be encoded in **Base64**.

Exploitation (Cookie Poisoning)

Step 1: Decode the Session Cookie

Copying the cookie string and decoding it using CyberChef with the `From Base64` operation reveals a plaintext JSON object containing user session data:

```
{ "user_id": 1, "username": "guest", "role": "guest", "email": "guest@techcorp.local" }
```

Step 2: Modify the Payload

Since the application relies entirely on client-side session data stored within the cookie without server-side cryptographic verification or signature checking (such as HMAC or JWT signing), we can tamper with the parameters.
We modify the JSON payload to elevate our privileges by changing the `username` and `role` fields to `admin`:

```
{ "user_id": 1, "username": "admin", "role": "admin", "email": "guest@techcorp.local" }
```

Step 3: Encode and Inject

1. Encode the modified JSON string back into **Base64** using CyberChef.
2. Replace the old session cookie value in the browser's Developer Tools with the newly generated Base64 string.
3. Refresh the page.

Upon reloading, the application parses the modified session cookie, grants administrative privileges, and reveals the flag.

Remediation & Prevention

To secure applications against cookie poisoning and insecure deserialization:

- **Server-Side Sessions:** Store session states securely on the server side (e.g., in a secure database or Redis store) and only issue a random, unpredictable session identifier (token) to the client.
- **Cryptographic Signing / Encryption:** If session data must reside on the client side, ensure it is cryptographically signed (using strong HMAC) or encrypted so that users cannot tamper with the attributes without invalidating the signature.